centro universitário FEI

CC1620 - Desenvolvimento Web
CCP120 - Programação Full Stack

Não esqueçam de marcar presença!

Linguagens de Programação

Linguagens de Programação

- As linguagens de programação são linguagens formais para comunicar instruções aos computadores.

Linguagens de Programação

- Algumas linguagens de programação:
- C
- C++
- Java
- C#
- Objective-C
- Assembly
- Python
- JavaScript
- Ruby
- Pearl
- PHP
- Matlab

Linguagens de Programação

- As linguagens de programação se dividem em 2 níveis:
  - Baixo
  - Alto

Linguagens de Programação

- Baixo: aquelas que oferecem pouca ou nenhuma abstração ao conjunto de instruções da arquitetura.
  - São rápidas
  - Não há necessidade de compilar, pois já está no formato que o processador “entende”

Linguagens de Programação

- Alto: Apresentam alta abstração do conjunto de instruções do computador. Neste tipo de linguagem não é necessário que se conheça a arquitetura do computador (32 / 64 bits; etc).
  - Um pouco mais lentas que a de baixo nível
  - Precisa de um compilador (que traduz o programa para uma linguagem de baixo nível)

Linguagens de Programação

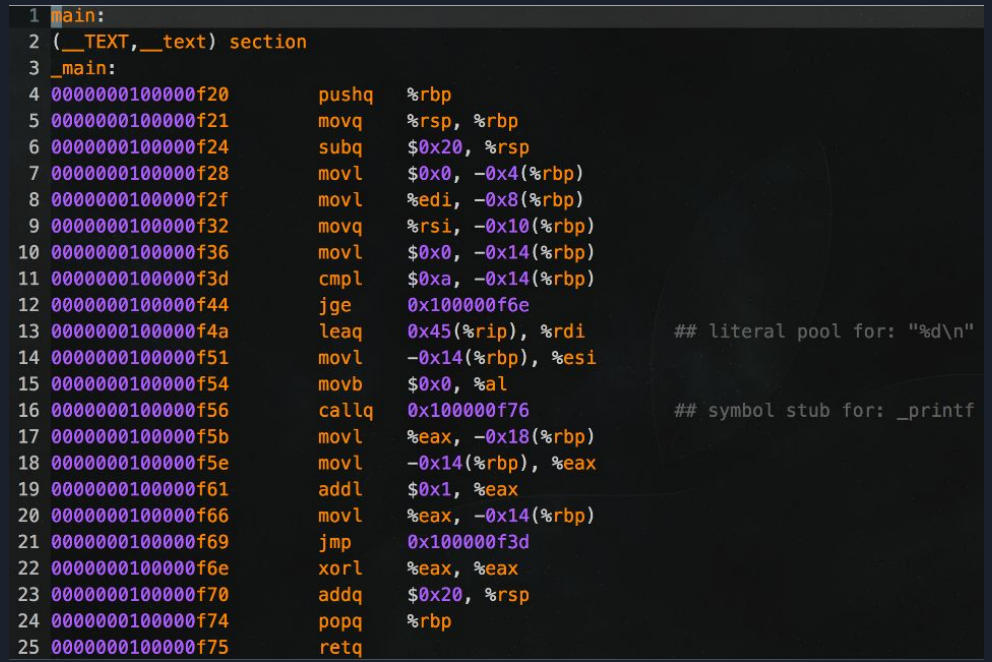
- Exemplo de linguagem de alto nível (*Python*)

```python
a = int(input("Primeiro Valor: "))
b = int(input("Segundo número: "))

if a > b:
    print("O primeiro é o maior!")
else:
    print("O segundo é o maior!")
```

Linguagens de Programação

- Exemplo de linguagem de baixo nível (*Assembly*)



```
1  main:
2  (__TEXT,__text) section
3  _main:
4  0000000100000f20        pushq   %rbp
5  0000000100000f21        movq    %rsp, %rbp
6  0000000100000f24        subq    $0x20, %rsp
7  0000000100000f28        movl    $0x0, -0x4(%rbp)
8  0000000100000f2f        movl    %edi, -0x8(%rbp)
9  0000000100000f32        movq    %rsi, -0x10(%rbp)
10 0000000100000f36        movl    $0x0, -0x14(%rbp)
11 0000000100000f3d        cmpl    $0xa, -0x14(%rbp)
12 0000000100000f44        jge     0x100000f6e
13 0000000100000f4a        leaq    0x45(%rip), %rdi        ## literal pool for: "%d\n"
14 0000000100000f51        movl    -0x14(%rbp), %esi
15 0000000100000f54        movb    $0x0, %al
16 0000000100000f56        callq   0x100000f76             ## symbol stub for: _printf
17 0000000100000f5b        movl    %eax, -0x18(%rbp)
18 0000000100000f5e        movl    -0x14(%rbp), %eax
19 0000000100000f61        addl    $0x1, %eax
20 0000000100000f66        movl    %eax, -0x14(%rbp)
21 0000000100000f69        jmp     0x100000f3d
22 0000000100000f6e        xorl    %eax, %eax
23 0000000100000f70        addq    $0x20, %rsp
24 0000000100000f74        popq    %rbp
25 0000000100000f75        retq
```

Linguagens de Programação

- Algumas linguagens frequentemente utilizadas para aplicações Web:
  - ***JavaScript***
  - Java
  - C#
  - PHP
  - Python

JavaScript



JavaScript



- Criada por Brendan Eich em 1995



Utilizada pela primeira vez dentro do navegador Nestcape 2.0 (chamava-se *LiveScript* nesta época)

- É uma linguagem de alto nível e interpretada
- Pode ser utilizada no Front-End e no Back-End



JavaScript



- Permite interação entre usuário e página
- *LiveScript* virou *JavaScript* na época em que a Netscape adicionou suporte à tecnologia Java em seu navegador.
- O nome causou (***e causa ainda***) confusão dando a impressão de que a linguagem foi baseada em Java.
- Muitos consideram a escolha do nome como uma estratégia de marketing da Netscape.

JavaScript



- Sintaxe parecida com C/C++
- Linguagem interpretada:
  - Quem é o interpretador??!!

JavaScript



- Sintaxe parecida com C/C++
- Linguagem interpretada:
  - Qual é o interpretador??!!
  - O Browser!

JavaScript

- O que é um Script?



JavaScript



- O que é um Script?

Script é um texto com uma série de instruções escritas para serem seguidas, ou por pessoas em peças teatrais ou programas televisivos, ou executadas por um programa de computador.

O termo é uma redução da palavra inglesa manuscript, que significa "manuscrito" ou "escrito à mão".

JavaScript

- O que é um Script?



Em Informática, script é um conjunto de instruções em código, ou seja, escritas em linguagem de computador.

Linguagens interpretadas são muitas vezes chamadas de *scripts*

JavaScript



- Como escrever Scripts em sua página?

```
<script type="text/javascript">
    // seu código vai aqui
</script>
```

- A tag <script> pode ser utilizada dentro das tags <body> ou <head>
- No <body>, os scripts devem ser adicionados na última linha

JavaScript



- *Internal call:*
  - script dentro do <body>
- *External call:*
  - script em um arquivo externo *“arquivo.js”*
  - Source (src) do script no fim do <body>
  - <script src=“arquivo.js” type=”text/javascript”></script>

JavaScript



- Para se declarar uma variável em JavaScript, pode-se utilizar as seguintes palavras-chaves:
  - *nenhuma palavra chave*
  - *var*,
  - *let* ou
  - *const*

Vamos ver isso em detalhes alguns slides para frente

Tipos das variáveis

- No *JavaScript* não se faz declaração explícita do tipo das variáveis
- Mas, mesmo assim, elas têm tipos!

JavaScript



- JavaScript possui 2 tipos nativos de dados:
  - Números reais ou inteiros:
  -

```
<script type="text/javascript">
    let x = 13.42;
</script>
```

  - String: cadeia de caracteres:

```
<script type="text/javascript">
    let x = "texto!";
</script>
```

Operadores aritméticos

+	adição
-	subtração
*	multiplicação
/	divisão
%	módulo / resto

Operadores Lógicos

&&	E condicional
||	Ou condicional
!	Não lógico

Conversões de tipos

- É possível convertermos variáveis do tipo *string* para variáveis numéricas:

```
let numero = "10";
numero = parseInt(numero);

let numero2 = "10.25";
numero2 = parseFloat(numero2);
```

- É possível convertermos variáveis do tipo numérico para *string*:

```
let texto = 2;
texto = String(texto);
```

JavaScript



- Formas de saída de dados no *JavaScript*

```
window.alert("olá!");
console.log("hello!");
```

- Forma de entrada de dados:

```
let entrada = prompt("Entre com o dado: ");
```

Controle de Fluxo / Estruturas Condicionais

```
if(entrada === 9){
    // faz algo
}
else if(entrada > 9){
    // caso entrada > 9 faz este bloco
}
else{
    // caso contrário
}
```

Igualdade Estrita *vs.* Igualdade Ampla

- Estrita === : Igualdade estrita compara dois valores para a igualdade. Nenhum valor é convertido implicitamente (semelhante ao == do Python)
- Ampla == : compara após converter ambos os valores para um tipo comum

Estruturas de Repetição - ***while***

```
let i = 0;

while(i<10){
    window.alert(i);
    i++;
}
```

Estruturas de Repetição - *for*

```
for(let i=0; i < 10; i++){
    window.alert(i);
}
```

Estruturas de Repetição - *for*

É executado (uma vez) antes da execução do bloco de código.

Condição para executar o código.

É executado (todas as vezes) após o bloco de código ter sido executado

```
for(let i=0; i < 10; i++){
    window.alert(i);
}
```

Estrutura do JavaScript dentro do HTML (*internal call*)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>JavaScript</title>
</head>
<body>
<script type="text/javascript">
    let entrada = prompt("Entre com o dado: ");
    window.alert("olá!");
    console.log("hello!");
</script>

</body>
</html>
```

JavaScript - Escopo das Variáveis



- Palavras-chaves:
  - *nenhuma palavra chave*
  - *var*,
  - *let* ou
  - *const*

Sem nenhuma palavra chave

- *variável global*

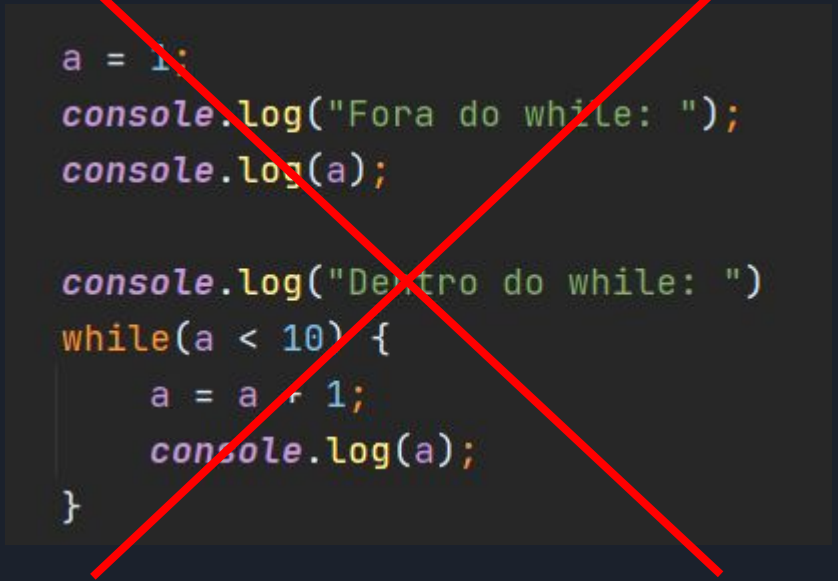
```
a = 1;
console.log("Fora do while: ");
console.log(a);

console.log("Dentro do while: ")
while(a < 10) {
    a = a + 1;
    console.log(a);
}
```

Escopo da variável a

Sem nenhuma palavra chave

- *variável global (não recomendado!)*



```
a = 1;
console.log("Fora do while: ");
console.log(a);

console.log("Dentro do while: ")
while(a < 10) {
    a = a + 1;
    console.log(a);
}
```

var

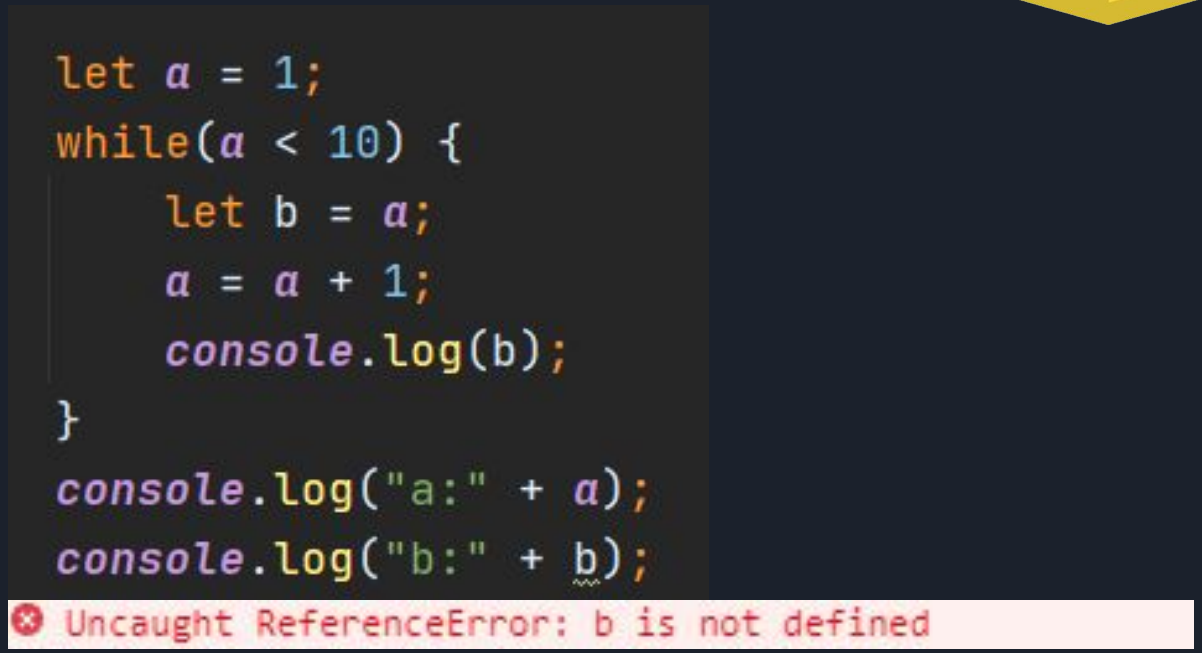
- *var - escopo de função*

```
var a = 1;
while(a < 10) {
    var b = a;
    a = a + 1;
    console.log(b);
}
console.log("a:" + a);
console.log("b:" + b);
```

Escopo da variável b

let

- ***let** - variável com escopo de bloco*



```
let a = 1;
while(a < 10) {
    let b = a;
    a = a + 1;
    console.log(b);
}
console.log("a:" + a);
console.log("b:" + b);
```

```
Uncaught ReferenceError: b is not defined
```

const

- *const - igual ao let, só que as variáveis são read-only*



```
1  const a = 1;
2  a = 2;
```

```
Uncaught TypeError: Assignment to constant variable.
```

Exercícios

1. Desenvolver um algoritmo em JavaScript que exiba os números ímpares de 1 a 100
2. Faça um programa para imprimir os múltiplos de 5, no intervalo de 1 até 500
3. Faça um programa em JavaScript que leia um número inteiro positivo e mostre todos os números a partir dele até zero (decrescente).

   Exemplo 1: numero = 8 -> Mostrar 8 7 6 5 4 3 2 1 0

   Exemplo 2: numero = 13 -> Mostrar 13 12 11 10 9 8 7 6 5 4 3 2 1 0

Exercícios

4. Desenvolver um algoritmo em JavaScript que exibe o resultado do fatorial de um número inteiro informado pelo usuário

   Exemplo: 5! = 5 x 4 x 3 x 2 x 1

   ou

   como na multiplicação a ordem dos fatores não altera o produto... 5! = 1 x 2 x 3 x 4 x 5

Exercícios

5. Foi realizada uma pesquisa entre os habitantes de uma dada região. Foram recolhidos os dados de idade, sexo (M/F) e salário. Construa um programa em JavaScript que informe:
    a. a média de salário do grupo;
    b. maior e menor idade do grupo;
    c. quantidade de mulheres com salário até R$5000,00.

Encerre a entrada de dados quando for digitada uma idade negativa.

Vá além!

Aprofunde seus conhecimentos em JavaScript

https://www.w3schools.com/js/default.asp

Não esqueçam de marcar presença!

Material desenvolvido por:

- Danilo Périco
- Isaac de Jesus
- Fagner Pimentel

Fontes:

- https://www.w3schools.com/